



UNIVERSIDAD
TECNOLÓGICA NACIONAL
FACULTAD REGIONAL
RESISTENCIA

PINKXEL

Intérprete Smart House y Traductor a HTML

Aguirre, Agustina Belen - 30082

Alberto, Angelina Lucia - 30088

Leiva, Sofia Victoria - 30059

Mujica, Ana Laura - 30071

Rodríguez, María Florencia - 30642

Verón, Milagros Luciana - 30782

Sintaxis y Semántica de los Lenguajes
Ingeniería en Sistemas de la Información

Primer Cuatrimestre
Curso académico: 2026

Universidad Tecnológica Nacional
Regional Resistencia

Resistencia, Chaco
7 de Junio de 2026

INDICE

INTRODUCCIÓN	1
CONFORMACIÓN DE GRUPO.....	2
INTEGRANTES.....	2
MATRIZ	2
IDENTIDAD.....	3
ALIANZA.....	4
GRAMÁTICA	5
ÁRBOL DE DERIVACIÓN	9
ANÁLISIS LÉXICO.....	10
OBJETIVOS	10
DISEÑO DEL LEXER	10
ARCHIVOS EN EL PROYECTO.....	11
<i>tokens.py</i>	12
<i>lexer.py</i>	12
<i>errores.py</i>	13
<i>main.py</i>	13
CONTROL DE ERRORES.....	14
CASOS DE PRUEBA Y RESULTADOS OBTENIDOS.....	14
DECISIONES DE DISEÑO	16
CONCLUSIONES	19



INTRODUCCIÓN

En esta segunda etapa del trabajo práctico integrador se desarrolló el analizador léxico (Lexer) correspondiente al lenguaje *SMART HOME*. El objetivo principal de este componente consiste en recibir una secuencia de caracteres ingresada por el usuario y transformarla en una serie de tokens válidos, los cuales representan las unidades léxicas fundamentales del lenguaje.

El análisis léxico constituye la primera fase dentro del proceso de compilación o interpretación de un lenguaje formal. Su función es identificar y clasificar los distintos elementos presentes en el código fuente, verificando además que estos respeten las reglas establecidas por la gramática definida para el lenguaje.

Para la implementación de esta etapa se utilizó el lenguaje de programación Python, desarrollando un conjunto de módulos capaces de reconocer palabras reservadas, sensores, actuadores, atributos, operadores, literales especiales y comentarios. Asimismo, se incorporaron mecanismos de validación que permiten detectar errores léxicos y proporcionar mensajes descriptivos al usuario, facilitando la identificación y corrección de problemas durante la escritura de programas *SMART HOME*.



CONFORMACIÓN DE GRUPO

Integrantes

- ❖ Aguirre, Agustina Belen - 30082
- ❖ Alberto, Angelina Lucia - 30088
- ❖ Leiva, Sofia Victoria - 30059
- ❖ Mujica, Ana Laura - 30071
- ❖ Rodríguez, María Florencia - 30642
- ❖ Verón, Milagros Luciana – 30782

Matriz

A continuación, se presenta la matriz de habilidades del equipo, confeccionada con el propósito de plantear las competencias clave para la elaboración del proyecto. En las columnas se encuentran las competencias, y en las filas cada integrante. En las celdas se indica el nivel de afinidad de cada integrante con la siguiente nomenclatura:

- ❖ ★: Experto
- ❖ ○: La persona no es experta, pero podría desenvolverse con dicha competencia
- ❖ Celda en blanco: Competencia donde la persona desconozca por completo

	Análisis de Datos/IA	Creatividad / Ideación de soluciones	Diseño	Edición de Multimedia	Gestión del Tiempo	Inglés	Investigación	Liderazgo / Coordinación	Logística de Bienestar	Normas Apa	Programación	Redacción y Ortografía
Aguirre, Agustina Belen	★	○	★	○		○	★	★	★	○	★	○
Alberto, Angelina Lucia	○	○	○	★		○	★	★	★	○	○	
Leiva, Sofia Victoria	○	★	★	○	○	○	○	○	○	★	○	★
Mujica, Ana Laura	○	★	★	★	○	★	○	○	○	★	○	★
Rodríguez, María Florencia	○	○	○			★	★	○	○	★	○	★
Verón, Milagros Luciana	★	★	★	○	○	★	★	★	○	○	○	★



Identidad

El nombre **PINKXEL** surge de la fusión de dos conceptos fundamentales: Pink y Pixel. Por un lado, el píxel representa la unidad mínima e indivisible de toda construcción digital. Por otro lado, Pink se presenta como un símbolo de elegancia, feminidad e intelecto, valores que atraviesan nuestra identidad como equipo; el color rosa representa una forma de pensar y construir, con sensibilidad en el diseño, precisión en la lógica y una mirada analítica que integra lo estético con lo funcional.

El logo del equipo, representado principalmente por un diamante, refuerza nuestra identidad, siendo este símbolo de nuestra esencia como grupo. El diamante representa resistencia, claridad y valor, cualidades que reflejan tanto el proceso de formación académica como el trabajo sostenido requerido para desarrollar soluciones tecnológicas de calidad. La elección de los colores rosa y negro acompaña esta construcción simbólica. El rosa aporta energía, creatividad e identidad, mientras que el negro representa solidez, firmeza y precisión.

Nuestra inspiración se encuentra principalmente en *Joan Clarke*, criptografía fundamental en el descifrado de códigos durante la Segunda Guerra Mundial, cuyo aporte fue durante mucho tiempo invisibilizado. Su figura representa la capacidad analítica, la resiliencia y el talento aplicado a problemas complejos, cualidades directamente vinculadas al diseño de analizadores léxicos y sintácticos.

A través de esta identidad, buscamos no solo representarnos como equipo, sino también aportar a la visibilización y fortalecimiento del rol de la mujer en la ingeniería. Esperamos sea de su agrado la propuesta y que el mensaje llegue de la mejor manera, mostrando el compromiso que tuvimos desde el día uno hasta la presentación final.



Alianza

Como equipo, nos comprometemos a trabajar de manera activa, responsable y colaborativa para alcanzar los objetivos del trabajo práctico. Cada integrante aportará desde sus habilidades, participando en las tareas asignadas (tanto grupales como individuales) y cumpliendo con los plazos acordados, organizando nuestros tiempos cuando sea necesario.

Nos proponemos mantener una comunicación constante y efectiva, estando disponibles tanto en encuentros presenciales como virtuales, respetando los horarios y responsabilidades individuales. Establecimos que nuestros medios de comunicación por donde realizaremos reuniones cada viernes con el fin de corroborar la evolución del proyecto y definir nuevos objetivos semanales serán mediante WhatsApp y Discord.

Asimismo, asumimos el compromiso de sostener el esfuerzo ante las dificultades, apoyándonos mutuamente y buscando resolver los problemas que surjan de la mejor manera posible. Como objetivo común, buscamos consolidar un trabajo que refleje el esfuerzo grupal y nos permita demostrar de forma clara lo que somos capaces de hacer como equipo.



GRAMÁTICA

En el siguiente apartado se presenta la gramática definida por el grupo a partir del escenario propuesto. Para su elaboración, se adoptó la convención de escribir los símbolos no terminales en minúscula y los terminales en mayúscula. Asimismo, se incorporaron comentarios en cada producción con el objetivo de describir brevemente su función dentro del lenguaje.

/*Símbolo Inicial*/

pinkxel → instruccion
| instruccion pinkxel

/*Instrucciones*/

instruccion → asignacion
| instruccion_if
| instruccion_every
| instruccion_when

/*Estructuras de Control*/

instruccion_when → WHEN condicion DO bloque END
instruccion_if → IF condicion THEN bloque END
| IF condicion THEN bloque ELSE bloque END
instruccion_every → EVERY tiempo DO bloque END

/*Bloque*/

bloque → instruccion
| instruccion bloque



/*Condiciones*/

condición → expresion

| expresion AND condicion

| expresion OR condicion

| NOT condición

/*Expresión*/

sensor_num → sensor_temp | sensor_humedad | sensor_luz

sensor_bool → sensor_movimiento | sensor_humo

expresion → expresion_num | expresion_bool | actuadores

expresion_num → sensor_num operador_comp valor

expresion_bool → sensor_bool operador_bool bool

/*Operadores*/

operador_comp → operador_bool

| <

| >

| <=

| >=

operador_bool → ==

| !=

/*Asignación*/

asignacion → actuadores =valor | actuadores



/*Valores*/

valor → bool

- | num tk_hora num
- | num tk_fecha num tk_fecha num
- | num tk_percent
- | num tk_grados

/*Booleanos*/

bool → tk_true

- | tk_false
- | tk_on
- | tk_off

/*Tiempo*/

tiempo → tk_num

- | tk_tiempo

/*Sensores*/

sensor → tk_SEN_HUMEDAD

- | tk_SEN_LUZ
- | tk_SEN_TEMPERATURA
- | tk_SEN_HUMO
- | tk_SEN_MOVIMIENTO

/*Delimitador*/

delimitador → .



/*Actuadores*/

actuadores → FOCO_ ID delimitador atributo_foco
 | AIRE_ ID delimitador atributo_aire
 | CERRADURA_ ID delimitador atributo_cerradura
 | PERSIANA_ ID delimitador atributo_persiana
 | ALARMA_ ID delimitador atributo_alarma
 | RELOJ_ ID delimitador atributo_reloj
 | ALTAVOZ_ ID delimitador atributo_altavoz

/*Atributos*/

atributo_foco → ESTADO operador_bool bool
 | BRILLO operador_comp valor
 | COLOR operador_comp valor

atributo_aire → ESTADO operador_bool bool
 | MODO operador_comp valor
 | TEMP_OBJ operador_comp valor
 | TEMP_ACT operador_comp valor

atributo_persiana → POSICION operador_comp valor

atributo_cerradura → ESTADO operador_bool bool

atributo_alarma → ESTADO operador_bool bool
 | ACTIVADA operador_bool bool



atributo_reloj → HORA operador_comp valor

| FECHA operador_comp valor

atributo_altavoz → VOLUMEN operador_comp valor

| MUTE operador_bool bool

| MENSAJE

| EMAIL

Árbol de Derivación

Para mejor comprensión de la gramática, el grupo propone un árbol de derivación, elaborado en base a un ejemplo.

EJEMPLO: Cuando el estado del foco sea encendido y la hora sean las 6 de la tarde, apagar el foco. Si la persiana está abierta un 35%, entonces cambiar la posición de la persiana a un 80%

WHEN FOCO.ESTADO = ON AND RELOJ.HORA = 18:00 DO FOCO.ESTADO = OFF IF PERSIANA.POSICION = 35% THEN PERSIANA.POSICION = 80%

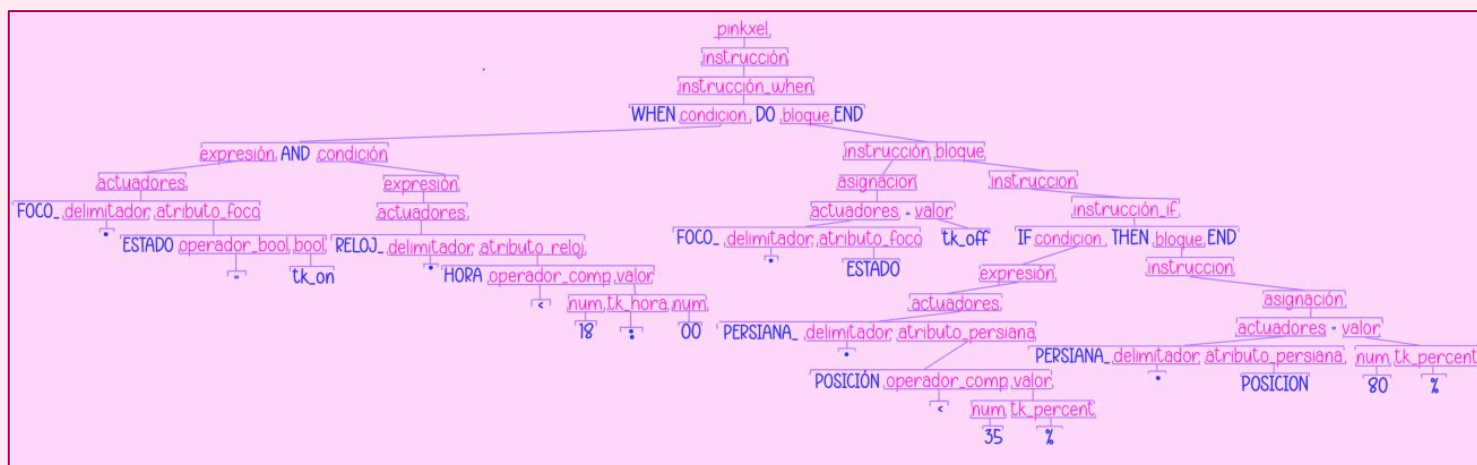


Figura 2. Árbol de derivación



ANÁLISIS LÉXICO

Objetivos

El objetivo general de esta etapa del proyecto consiste en **diseñar e implementar un analizador léxico** para el lenguaje **SMART HOME**, capaz de reconocer los componentes definidos por su gramática y transformarlos en tokens válidos para etapas posteriores del procesamiento.

Como objetivos específicos se establecieron:

- ❖ Implementar un mecanismo de *tokenización* para las instrucciones del lenguaje.
- ❖ Reconocer palabras reservadas, operadores, sensores, actuadores, atributos y literales especiales.
- ❖ Detectar errores léxicos y proporcionar mensajes descriptivos al usuario.
- ❖ Organizar la solución mediante una arquitectura modular que facilite futuras ampliaciones.
- ❖ Generar una base sólida para la posterior implementación del analizador sintáctico.

Diseño del Lexer

El analizador léxico fue diseñado siguiendo una estructura modular con el objetivo de facilitar el mantenimiento del código, mejorar su legibilidad y simplificar futuras ampliaciones del lenguaje. Para ello, las distintas responsabilidades del sistema fueron distribuidas en módulos independientes que colaboran entre sí durante el proceso de análisis.

El funcionamiento general del lexer comienza cuando el usuario ingresa una línea de código escrita en el lenguaje **SMART HOME**. Antes de realizar cualquier clasificación, el sistema **verifica** que todos los caracteres presentes pertenezcan al alfabeto permitido por el lenguaje. Esta validación inicial permite detectar símbolos inválidos y evitar que continúen participando en etapas posteriores del análisis.



Una vez validada la entrada, se lleva a cabo el proceso de **tokenización**. Durante esta etapa la línea es recorrida carácter por carácter con el fin de separar correctamente cada uno de los elementos léxicos presentes. Este procedimiento contempla situaciones particulares como cadenas delimitadas por comillas, comentarios y expresiones que contienen símbolos especiales.

Posteriormente, cada token obtenido es sometido a un proceso de clasificación. Para ello se verifica si corresponde a una palabra reservada, un operador, un sensor, un actuador, un atributo o alguno de los literales especiales definidos por el lenguaje. En caso de que un token no pueda ser reconocido, el sistema intenta determinar el tipo de error más probable para proporcionar un mensaje de diagnóstico adecuado.

Además del reconocimiento de tokens convencionales, el lexer implementa validaciones específicas para elementos propios del dominio de automatización residencial planteado por **SMART HOME**. Entre ellas se encuentran la validación de horarios, fechas, temperaturas, porcentajes, direcciones de correo electrónico y valores de iluminación expresados en lux.

Otra característica relevante del diseño consiste en el soporte para la notación **actuador.atributo**, utilizada para representar propiedades específicas de los distintos dispositivos de la vivienda inteligente. Esta funcionalidad permite separar correctamente cada componente y clasificarlos de manera independiente, facilitando el posterior análisis sintáctico.

Archivos en el proyecto

Con el propósito de mantener una arquitectura clara y organizada, el proyecto fue dividido en varios módulos independientes. Cada uno de ellos cumple una función específica dentro del proceso de análisis léxico, permitiendo separar responsabilidades y reducir el acoplamiento entre componentes.

Esta organización modular simplifica tanto el desarrollo como las tareas de mantenimiento y depuración, ya que cada archivo puede ser analizado y



modificado de manera independiente sin afectar significativamente al resto del sistema.

tokens.py

El archivo contiene todas las **definiciones relacionadas con los componentes léxicos** reconocidos por el lenguaje **SMART HOME**. Su función principal consiste en centralizar la información correspondiente a las distintas categorías de tokens para evitar duplicaciones y facilitar modificaciones futuras.

Dentro de este módulo se encuentran definidas las palabras reservadas, operadores relacionales y de asignación, símbolos especiales, sensores, actuadores, atributos y valores booleanos. Gracias a esta estructura, cualquier cambio en la especificación del lenguaje puede realizarse modificando únicamente este archivo, sin necesidad de alterar la lógica de procesamiento implementada en el lexer.

lexer.py

El módulo **lexer.py** constituye el núcleo principal del proyecto. En él se implementan todas las **funciones** necesarias para realizar el análisis léxico de las líneas de código ingresadas por el usuario.

Entre sus responsabilidades principales se encuentran la verificación de caracteres válidos, la tokenización de las líneas de entrada, la clasificación de tokens y la validación de distintos formatos especiales definidos por la gramática del lenguaje.

Asimismo, este módulo incorpora funciones específicas para reconocer dispositivos domóticos. Los sensores fueron divididos en sensores numéricos y sensores booleanos, lo que permitirá en futuras etapas verificar la compatibilidad entre operadores y tipos de datos. De forma similar, los actuadores fueron implementados mediante un esquema basado en prefijos e identificadores,



permitiendo representar múltiples instancias de un mismo tipo de dispositivo, como por ejemplo `foco_cocina`, `foco_living` o `aire_dormitorio`.

errores.py

El archivo `errores.py` agrupa todas las funciones destinadas a la generación de mensajes de error. Cada función se encuentra especializada en un tipo particular de error léxico y tiene como objetivo proporcionar información clara acerca de la naturaleza del problema detectado.

Entre los errores contemplados se encuentran los símbolos inválidos, correos electrónicos mal formados, horarios incorrectos, fechas inválidas, temperaturas fuera del formato esperado, porcentajes fuera de rango, cadenas mal delimitadas y tokens desconocidos.

La separación de esta funcionalidad en un módulo independiente permite mantener el código del lexer más limpio y facilita la modificación futura del formato de los mensajes sin alterar la lógica principal del análisis.

main.py

El archivo `main.py` actúa como punto de entrada de la aplicación y coordina la interacción entre el usuario y los distintos módulos del sistema.

Su funcionamiento consiste en mostrar una interfaz básica de consola, solicitar líneas de código **SMART HOME**, invocar los procesos de validación y tokenización, clasificar los tokens obtenidos y presentar los resultados correspondientes. Además, este módulo administra la numeración de líneas ingresadas por el usuario, información que posteriormente es utilizada para generar mensajes de error más precisos y facilitar el proceso de depuración.

La implementación de este archivo permite integrar todos los componentes desarrollados durante esta etapa y proporciona una interfaz simple para verificar el correcto funcionamiento del analizador léxico.



Control de errores

Por su parte en el trabajo se implementó un mecanismo de validación orientado a detectar errores asociados tanto a la estructura de los tokens como al formato de determinados literales definidos por el lenguaje *SMART HOME*.

Durante el análisis de una línea de entrada, el sistema verifica en primer lugar que **todos los caracteres utilizados pertenezcan al alfabeto permitido**. Esta comprobación inicial permite identificar símbolos inválidos que no forman parte de la especificación del lenguaje. Posteriormente, **cada token reconocido es sometido a un conjunto de validaciones específicas** según su categoría léxica.

Entre los controles implementados se encuentran la validación de direcciones de correo electrónico, horarios, fechas, porcentajes, temperaturas y cadenas de texto. De esta forma, no solo se verifica que un token pertenezca a una determinada categoría, sino también que respete las restricciones sintácticas establecidas para dicha categoría.

Con el propósito de facilitar la depuración de programas, **cada error detectado es acompañado por un mensaje descriptivo** que indica la línea en la que ocurrió y el elemento responsable del problema. Esta estrategia permite que el usuario identifique rápidamente la causa del error y realice las correcciones necesarias.

Casos de prueba y resultados obtenidos

Una vez finalizada la implementación del analizador léxico, se llevó a cabo un conjunto de pruebas destinadas a verificar el correcto reconocimiento de los distintos componentes definidos por la gramática del lenguaje *SMART HOME*. **Estas pruebas incluyeron tanto entradas válidas como situaciones diseñadas específicamente para provocar errores léxicos.**

Las verificaciones realizadas permitieron comprobar el reconocimiento correcto de palabras reservadas, sensores, actuadores, atributos, operadores y literales especiales. Asimismo, se evaluó el comportamiento del sistema frente a



cadena de texto, comentarios y expresiones que contienen símbolos especiales, con el objetivo de garantizar que la tokenización se efectuara de manera adecuada en todos los casos contemplados por la especificación del lenguaje.

De forma complementaria, se ejecutaron pruebas orientadas a validar los mecanismos de detección de errores implementados. Para ello se utilizaron ejemplos con formatos incorrectos de fechas, horarios, correos electrónicos, temperaturas y porcentajes, verificando que el sistema fuera capaz de identificar cada situación y emitir el mensaje correspondiente.

Los resultados obtenidos demostraron que el lexer desarrollado cumple con los requisitos establecidos para esta etapa del proyecto. En todos los casos analizados se logró una correcta clasificación de los tokens reconocidos, así como una adecuada detección e identificación de las distintas situaciones de error consideradas durante el diseño.

Tabla 1. Casos de prueba para tokens válidos

Caso Evaluado	Entrada	Resultado Esperado
Palabra Reservada	WHEN	TOKEN_WHEN
Sensor	sensor_luz	TOKEN_SENSOR_LUZ
Actuador con Atributo	reloj_cocina.HORA	TOKEN_ACTUADOR, TOKEN_PUNTO, TOKEN_HORA
Temperatura	25°C	TOKEN_TEMPERATURA
Porcentaje	80%	TOKEN_PORCENTAJE
Cadena de Texto	"Buenas Noches"	TOKEN CADENA
Comentario	// Apaga las luces	TOKEN_COMENTARIO



Tabla 2. Casos de prueba para errores léxicos

Caso Evaluado	Entrada	Resultado Esperado
Hora inválida	25:90	Error Léxico: Hora inválida
Fecha inválida	31/15/2026	Error Léxico: Fecha inválida
Email inválido	usuario@@gmail.com	Error Léxico: email inválido
Porcentaje fuera del rango	150%	Error Léxico: porcentaje inválido
Cadena sin cerrar	"Buenas Noches	Error Léxico: cadena mal delimitada
Token desconocido	&sensor	Error Léxico: token desconocido

Decisiones de diseño

Durante el desarrollo del analizador léxico se tomaron diversas decisiones de diseño orientadas a mejorar la organización del proyecto y facilitar futuras ampliaciones del lenguaje.

En primer lugar, se optó por una arquitectura modular basada en la separación de responsabilidades. Como consecuencia, las definiciones de tokens, la lógica de análisis léxico, el manejo de errores y la interacción con el usuario fueron implementados en archivos independientes. Esta estrategia favorece la reutilización del código y simplifica las tareas de mantenimiento.

Durante el desarrollo del analizador léxico se evaluó la posibilidad de utilizar librerías especializadas para el reconocimiento de patrones y el procesamiento de cadenas de texto. Sin embargo, se optó por implementar manualmente la mayor parte de las funcionalidades del lexer utilizando únicamente herramientas básicas proporcionadas por el lenguaje Python.



Esta decisión tuvo como objetivo comprender en profundidad el funcionamiento interno de un analizador léxico y desarrollar de forma explícita cada una de las etapas involucradas en el proceso de reconocimiento de tokens. En lugar de delegar estas tareas a bibliotecas externas, se implementaron **funciones propias** para la validación de correos electrónicos, fechas, horarios, temperaturas, porcentajes y otros literales definidos por la gramática del lenguaje **SMART HOME**.

De manera similar, el proceso de **tokenización fue desarrollado específicamente para los requerimientos del proyecto**. Aunque existen librerías capaces de dividir texto y reconocer patrones complejos, se decidió recorrer cada línea carácter por carácter para controlar situaciones particulares del lenguaje, tales como el reconocimiento de cadenas entre comillas, comentarios y expresiones del tipo **actuador.atributo**.

Así, se implementaron mecanismos propios para la clasificación de tokens mediante estructuras de datos y funciones de validación, evitando depender de herramientas externas de análisis léxico. Este enfoque permitió adaptar completamente el comportamiento del lexer a las necesidades específicas del lenguaje diseñado y facilitó la comprensión de los conceptos estudiados en la asignatura.

La utilización de funciones desarrolladas por el propio equipo contribuyó además a mantener un mayor control sobre el funcionamiento del sistema, simplificando las tareas de depuración y permitiendo justificar con claridad cada una de las decisiones de diseño adoptadas durante la implementación.

Asimismo, **los sensores fueron clasificados en sensores numéricos y sensores booleanos**. Esta distinción permitirá, en etapas posteriores del proyecto, realizar validaciones sintácticas y semánticas más precisas, garantizando que las operaciones realizadas sean compatibles con el tipo de dato producido por cada sensor.



Por otra parte, se decidió representar los dispositivos mediante una estructura compuesta por un prefijo y un identificador. Gracias a este enfoque, el lenguaje puede modelar múltiples instancias de un mismo tipo de actuador, como `foco_cocina`, `foco_living` o `aire_dormitorio`, manteniendo una sintaxis uniforme y fácilmente extensible.

Finalmente, se optó por tokenizar la expresión `actuador.atributo` separando cada uno de sus componentes en tokens independientes. De esta manera, una expresión como `reloj_cocina.HORA` es interpretada como `TOKEN_ACTUADOR`, `TOKEN_PUNTO` y `TOKEN_HORA`. Esta decisión simplifica el análisis sintáctico posterior y proporciona una representación más consistente de la estructura interna del lenguaje.



CONCLUSIONES

El desarrollo del **analizador léxico** permitió aplicar de manera práctica los conceptos estudiados en la asignatura relacionados con lenguajes formales, gramáticas y procesos de compilación.

A lo largo de esta etapa se logró implementar un lexer funcional capaz de reconocer los distintos componentes definidos por la gramática del lenguaje **SMART HOME**, realizar el proceso de **tokenización** y **detectar** diversas situaciones de **error**. Asimismo, se incorporaron mecanismos de **validación** específicos para los elementos propios del dominio de automatización residencial planteado por el proyecto.

La utilización de una **arquitectura modular** permitió mantener una adecuada separación de responsabilidades entre los distintos componentes del sistema, favoreciendo la legibilidad del código y facilitando futuras tareas de mantenimiento y ampliación.

Los resultados obtenidos durante las pruebas realizadas permitieron verificar el correcto funcionamiento de las funcionalidades implementadas, constituyendo una base sólida para la siguiente etapa del trabajo, correspondiente al desarrollo del analizador sintáctico y de los mecanismos de validación semántica del lenguaje.

